

UNIVERSITÉ DE YAOUNDÉ 1

INF4027
GÉNIE LOGICIEL

Rapport Test logiciel et Model checking

Groupe 1 :

TOUKAP NGANSOP Rayan Ledoux 22U2142

NINGAHE SIMON Pierre 21T2832

DJIMELI TCHINDA Billyna Ariane 22W2171

MIMCHE MOLUH Ishak Nazir 21T2466

FOMPOUPASSE MBOUOMBOUO MOHAMED Faras 2U2061

Enseignant :

Pr ATSA ETOUNDI ROGER



Table des matières

Introduction	2
1 Analyse du problème	2
1.1 Contexte	2
1.2 Problématique	2
1.3 Objectifs	2
1.4 Contraintes	2
2 Étude théorique / État de l'art	2
2.1 Tests logiciels	2
2.1.1 Définition	2
2.1.2 Types de tests	3
2.1.3 Limites	3
2.2 Model Checking	3
2.2.1 Définition	3
2.2.2 Avantages	3
2.2.3 Limites	3
2.2.4 Outils connus	3
2.2.5 Logique Temporelle	3
3 Conception et répartition des tâches	4
3.1 Organisation du groupe	4
3.2 Diagramme général	4
3.3 Méthodologie	4
4 Réalisation	4
4.1 Exemple de test logiciel	4
4.2 Exemple de model checking avec NuSMV	5
5 Résultats et Discussion	5
5.1 Résultats obtenus	5
5.2 Comparaison	6
5.3 Discussion	6
6 Conclusion	6
7 References	6
8 Annexes	6



Introduction

Les systèmes logiciels modernes sont de plus en plus complexes, intégrés et critiques. Assurer leur fiabilité est donc devenu indispensable. Deux approches complémentaires sont principalement utilisées :

Les tests logiciels, qui permettent de détecter des erreurs en exécutant le programme.

Le model checking, une méthode de vérification formelle qui explore automatiquement tous les états possibles d'un système modélisé.

Ce rapport vise à étudier ces deux méthodologies, leurs principes, leurs forces et leurs limites, ainsi que leurs domaines d'application.

1 Analyse du problème

1.1 Contexte

Les logiciels étant omniprésents, leur mauvaise exécution peut entraîner des conséquences graves (pertes financières, sécurité compromise, etc.).

1.2 Problématique

Comment garantir la qualité, la correction, et la fiabilité d'un logiciel malgré sa complexité croissante ?

1.3 Objectifs

- Présenter les tests logiciels.
- Introduire le model checking.
- Comparer les deux approches.
- Illustrer leur utilisation à travers de petits exemples.

1.4 Contraintes

- Le model checking souffre du problème d'explosion combinatoire.
- Les tests ne garantissent jamais l'absence d'erreurs.

2 Étude théorique / État de l'art

2.1 Tests logiciels

2.1.1 Définition

Le test logiciel est un processus visant à détecter des erreurs en exécutant un programme dans des conditions contrôlées.



2.1.2 Types de tests

- Tests unitaires
- Tests d'intégration
- Tests fonctionnels
- Tests système
- Tests de régression

2.1.3 Limites

Ne couvre jamais tous les cas.
Dépend fortement de la qualité des scénarios.

2.2 Model Checking

2.2.1 Définition

Le model checking est une technique de vérification formelle qui consiste à :

1. Modéliser le système (automate, réseau de Petri...)
2. Exprimer les propriétés (CTL, LTL)
3. Explorer automatiquement tous les états

2.2.2 Avantages

- Vérification exhaustive
- Détection automatique des contre-exemples
- Très efficace pour les systèmes critiques

2.2.3 Limites

- Explosion d'états
- Nécessite une modélisation précise
- Demande des compétences en logique temporelle

2.2.4 Outils connus

- SPIN
- NuSMV / nuXmv
- UPPAAL
- PRISM

2.2.5 Logique Temporelle

En vérification formelle, on utilise trois logiques temporelles principales pour exprimer des propriétés sur l'évolution d'un système : **LTL (Linear Temporal Logic)** décrit le comportement le long de chemins linéaires dans le temps, en permettant de dire par exemple "toujours que x est vrai, y sera vrai à l'étape suivante" ; **CTL (Computation Tree Logic)** considère tous les chemins possibles à partir d'un état et permet de formuler des propriétés comme "il existe un chemin où éventuellement y devient vrai" ; enfin, **CTL*** combine la puissance de LTL et CTL, permettant à la fois des expressions sur des chemins

spécifiques et sur tous les chemins possibles, offrant ainsi une logique plus expressive pour vérifier des systèmes complexes.

3 Conception et répartition des tâches

3.1 Organisation du groupe

- Ningahe : Recherche sur les tests unitaires et rédaction du chapitre 2
- Mimche : Recherche sur le model checking et expérimentation avec SPIN
- Djimeli : Implémentation et tests pratiques
- Fompoupasse : Analyse des résultats et rédaction du chapitre 5
- Toukap : Conception générale et mise en forme du rapport

3.2 Diagramme général

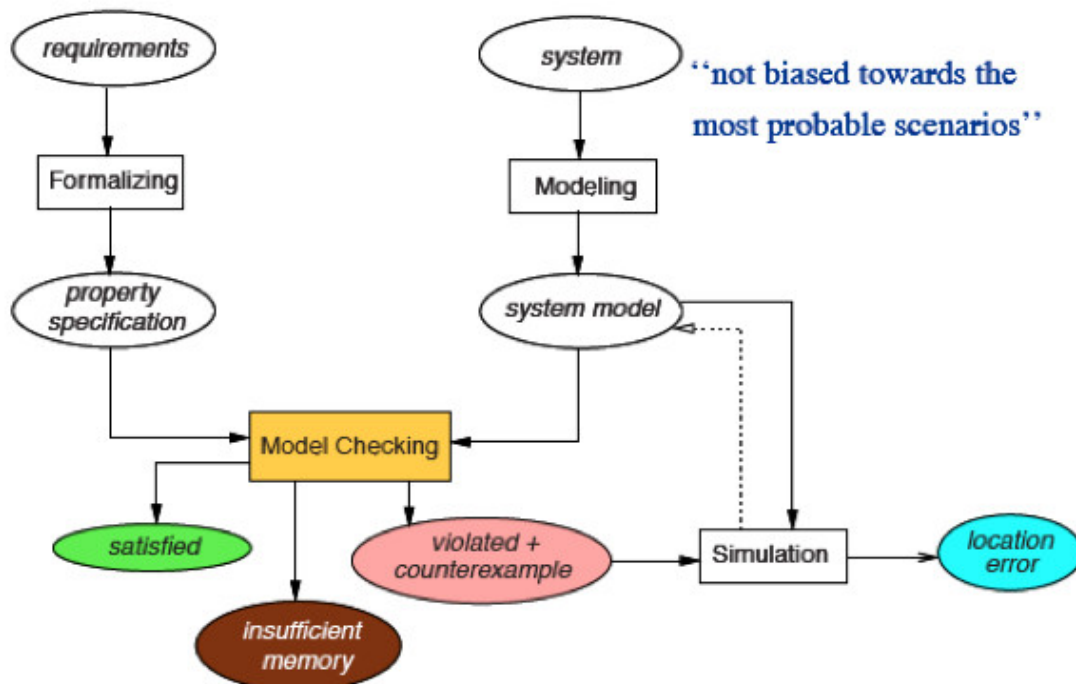


FIGURE 1 – Illustration du model checking

3.3 Méthodologie

Mix de démarche expérimentale et étude documentaire..

4 Réalisation

4.1 Exemple de test logiciel

Supposons une fonction qui vérifie si un nombre est pair :



```
def est_pair(n):  
    return n % 2 == 0
```

Test unitaire :

```
def test_est_pair():  
    assert est_pair(4) == True  
    assert est_pair(7) == False
```

4.2 Exemple de model checking avec NuSMV

Modèle (machine à états) :

```
MODULE main  
VAR  
    x : boolean;  
    y : boolean;  
  
ASSIGN  
    init(x) := TRUE;  
    next(x) := !x;  
  
    init(y) := FALSE;  
    next(y) := x;
```

Propriété à vérifier (LTL)

« Il est toujours vrai que si x est vrai, y deviendra vrai au prochain état »

LTLSPEC G (x $\rightarrow$ X y)

NuSMV explore tous les états et confirme ou infirme la propriété.

5 Résultats et Discussion

5.1 Résultats obtenus

- Les tests logiciels ont permis de valider le fonctionnement de petites fonctions.
- Le model checking a confirmé ou infirmé des propriétés logiques complexes.

5.2 Comparaison

Critère	Tests logiciels	Model checking
Couverture des cas	limitée	exhaustive
Complexité	faible	élevée
Facilité d'utilisation	simple	nécessite formation
Détection des erreurs	oui	oui + contres-exemples

TABLE 1 – Tableau comparatif

5.3 Discussion

Les deux approches sont complémentaires :

- Les tests : pratiques et rapides
- Le model checking : rigoureux et exhaustif

6 Conclusion

Ce rapport a permis d'explorer deux méthodes essentielles pour assurer la qualité logicielle. Les tests logiciels offrent une validation pragmatique, tandis que le model checking fournit une garantie formelle de correction. Leur combinaison est particulièrement recommandée pour les systèmes critiques.

7 References

1. Wikipedia
2. Documentation SPIN.
3. SPIN Model Checker – Official Manual.

8 Annexes

- Code complet SPIN nanoPromela avec une propriété fausse



```
int x = 0;

proctype Inc() {
do
:: true ->
if
:: (x < 200) -> x = x + 1
:: else -> skip
fi
od
}

proctype Dec() {
do
:: true ->
if
:: (x > 0) -> x = x - 1
:: else -> skip
fi
od
}

proctype Reset() {
do
:: true ->
if
:: (x == 200) -> x = 0
:: else -> skip
fi
od
}

init {
run Inc();
run Dec();
run Reset()
}

/* Propriété 1: x reste toujours entre 210 et 400 */
ltl p1 { [] (x >= 210 && x <= 400) }

/* Propriété 2: x atteint éventuellement 200 */
ltl p2 { <> (x == 200) }

/* Propriété 3: Si x atteint 200, il reviendra à 0 */
ltl p3 { [] (x == 200 -> <> (x == 0)) }
```

– Captures d’écran du test nanoPromela



```
rayan@rayan-Latitude-3310:~$ ./pan
warning: never claim + accept labels requires -a flag to fully verify
pan: ltl formula p1
pan:1: assertion violated !( (((x>=210)&&(x<=400)))) (at depth 0)
pan: wrote test.pml.trail

(Spin Version 6.5.2 -- 6 December 2019)
Warning: Search not completed
       + Partial Order Reduction

Full statespace search for:
    never claim           + (p1)
    assertion violations + (if within scope of claim)
    acceptance  cycles  - (not selected)
    invalid end states  - (disabled by never claim)

State-vector 28 byte, depth reached 0, errors: 1
    1 states, stored
    0 states, matched
    1 transitions (= stored+matched)
    0 atomic steps
hash conflicts:          0 (resolved)

Stats on memory usage (in Megabytes):
    0.000    equivalent memory usage for states (stored*(State-vector + overhead))
    0.286    actual memory usage for states
   128.000    memory used for hash table (-w24)
    0.534    memory used for DFS stack (-m10000)
   128.730    total actual memory usage

pan: elapsed time 0 seconds
```

– Code propre complet SPIN nanoPromela

```
int x = 0;

proctype Inc() {
do
  :: true ->
    if
      :: (x < 200) -> x = x + 1
      :: else -> skip
    fi
od
}

proctype Dec() {
do
  :: true ->
    if
      :: (x > 0) -> x = x - 1
      :: else -> skip
    fi
od
}

proctype Reset() {
do
  :: true ->
    if
      :: (x == 200) -> x = 0
      :: else -> skip
    fi
od
}

init {
  run Inc();
  run Dec();
  run Reset()
}

/* Propriété 1: x reste toujours entre 0 et 200 */
ltl p1 { [] (x >= 0 && x <= 200) }

/* Propriété 2: x atteint éventuellement 200 */
ltl p2 { <> (x == 200) }

/* Propriété 3: Si x atteint 200, il reviendra à 0 */
ltl p3 { [] (x == 200 -> <> (x == 0)) }
```

– Captures d’écran du test nanoPromela



```
rayan@rayan-Latitude-3310:~$ ./pan

(Spin Version 6.5.2 -- 6 December 2019)
+ Partial Order Reduction

Full statespace search for:
  never claim           - (none specified)
  assertion violations   +
  acceptance cycles     - (not selected)
  invalid end states     +

State-vector 44 byte, depth reached 4813, errors: 0
  8243 states, stored
  11647 states, matched
  19890 transitions (= stored+matched)
  0 atomic steps
hash conflicts:          10 (resolved)

Stats on memory usage (in Megabytes):
  0.566      equivalent memory usage for states (stored*(State-vector + overhead))
  0.582      actual memory usage for states
 128.000     memory used for hash table (-w24)
  0.534      memory used for DFS stack (-m10000)
 129.022     total actual memory usage

unreached in proctype Inc
  test.pml:11, state 11, "-end-"
  (1 of 11 states)
unreached in proctype Dec
  test.pml:21, state 11, "-end-"
  (1 of 11 states)
unreached in proctype Reset
  test.pml:31, state 11, "-end-"
  (1 of 11 states)
unreached in init
  (0 of 4 states)

pan: elapsed time 0 seconds
rayan@rayan-Latitude-3310:~$ # Génération avec la propriété p1
```